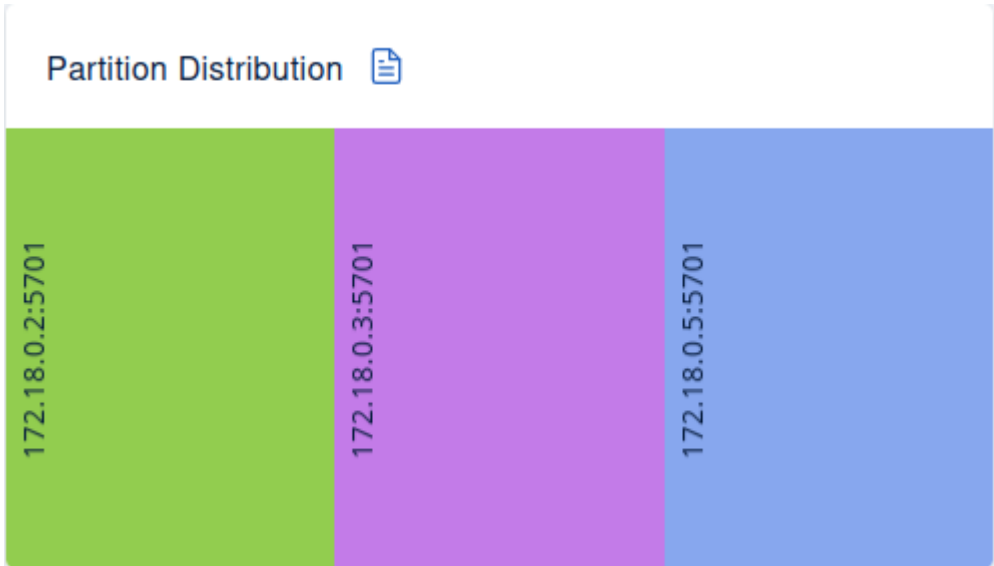


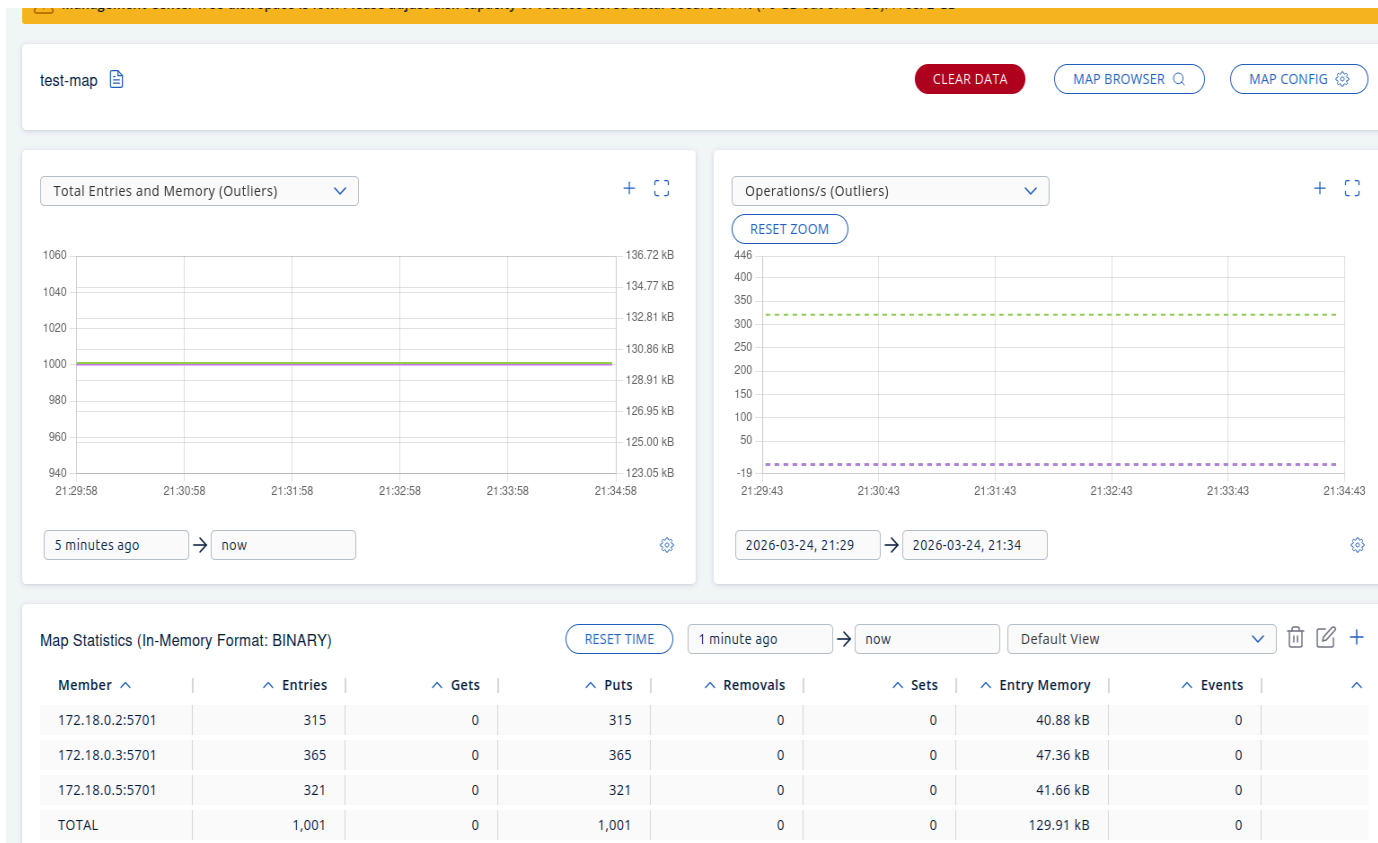
Here are the 3 Hazelcast nodes running in a cluster.



Resources Utilization 

Member ^	^ CPU	^ Used ...	^ Total ...	^ Heap ...
172.18.0.2:5701	20.46 %	64.83 MB	360.00 MB	18.01 %
172.18.0.3:5701	21.30 %	62.44 MB	546.00 MB	11.44 %
172.18.0.5:5701	20.79 %	68.66 MB	410.00 MB	16.75 %

```
(.venv) bubuntu@bubuntu-computer:~/CLionProjects/Microservices$  
python task3_populate.py  
Successfully inserted 1000 elements.
```



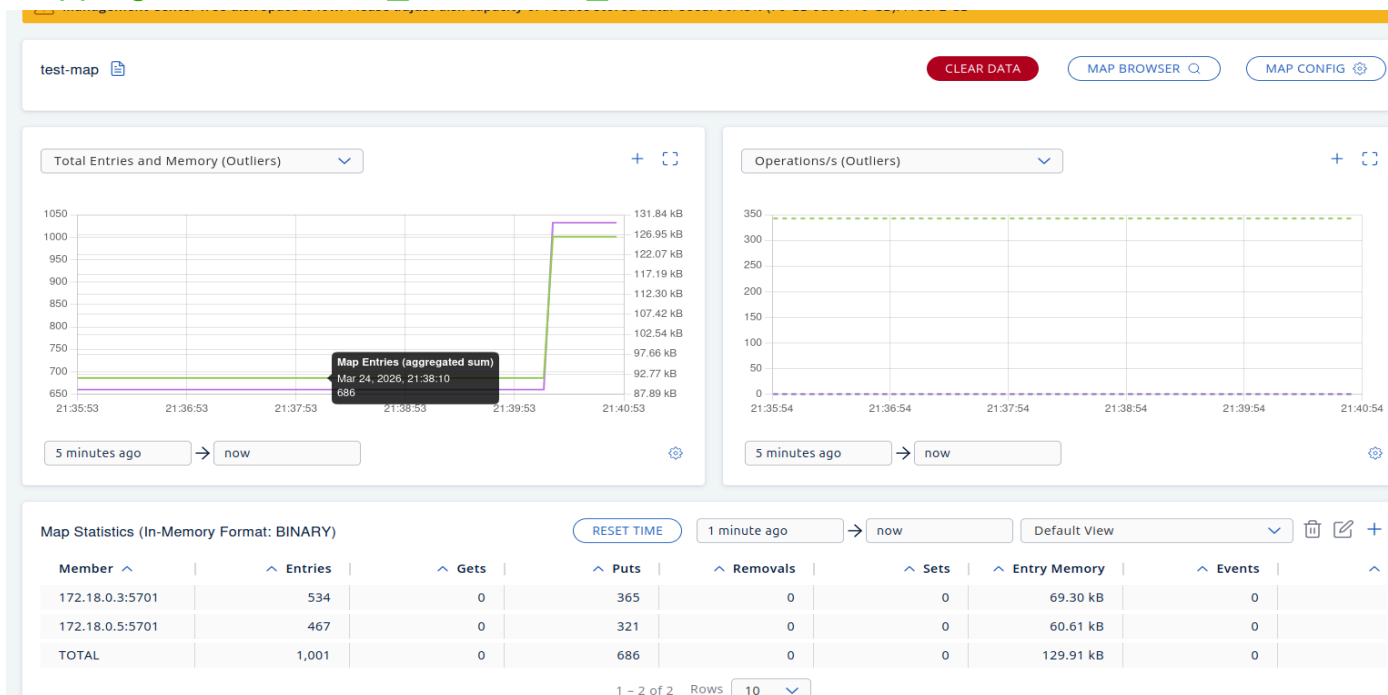
We can see that they are evenly distributed between 3 nodes

Deleting 1 node

(.venv) bubuntu@bubuntu-computer:~/CLionProjects/Microservices\$

docker-compose stop hazelcast1

Stopping microservices_hazelcast1_1 ... done



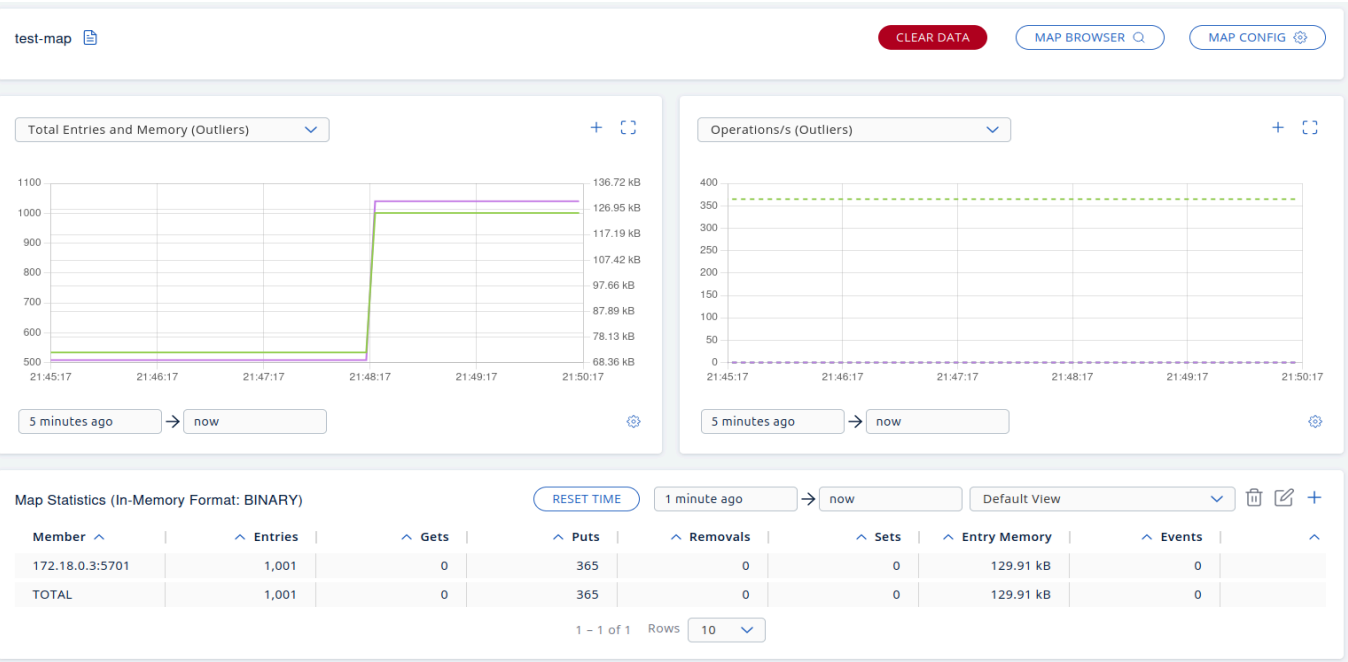


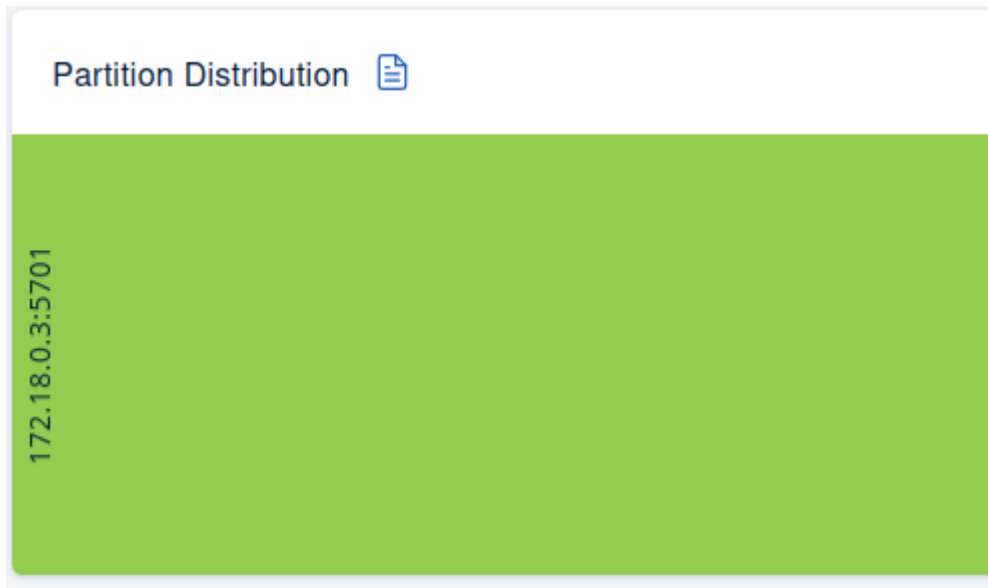
Let's stop next node

(.venv) bubuntu@bubuntu-computer:~/CLionProjects/Microservices\$

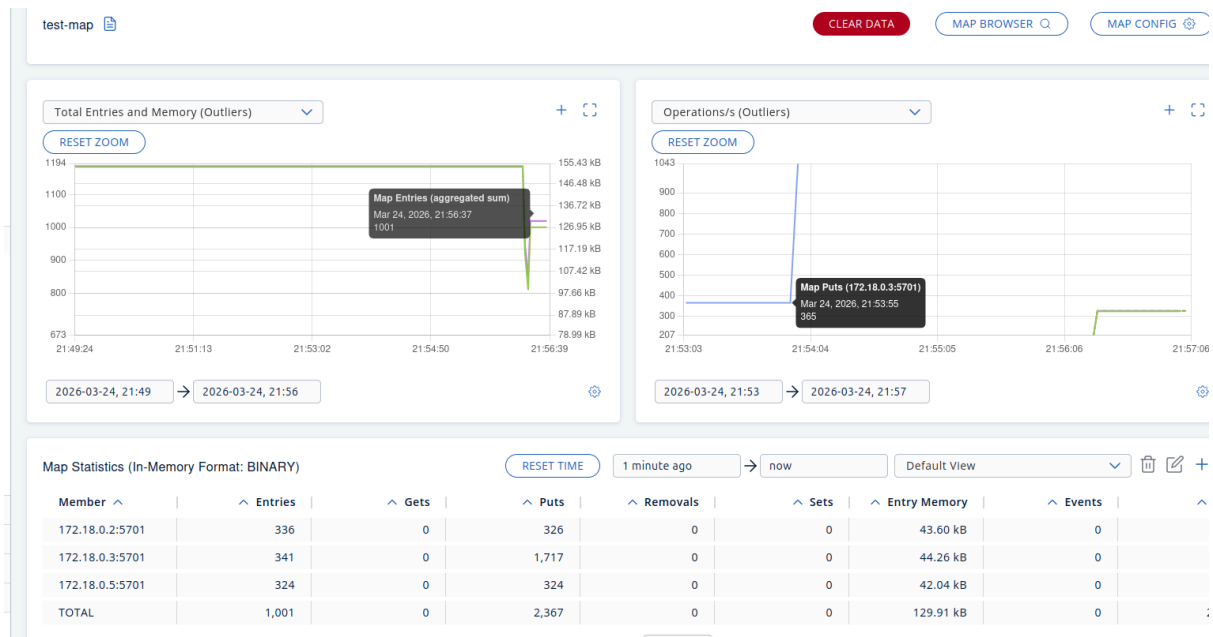
docker-compose stop hazelcast2

Stopping microservices_hazelcast2_1 ... done





Now i repopulated the nodes:



Partition Distribution



```
(.venv) bubuntu@bubuntu-computer:~/CLionProjects/Microservices$
docker-compose kill hazelcast1 hazelcast2
Killing microservices_hazelcast1_1 ... done
Killing microservices_hazelcast2_1 ... done
```

Partition Distribution

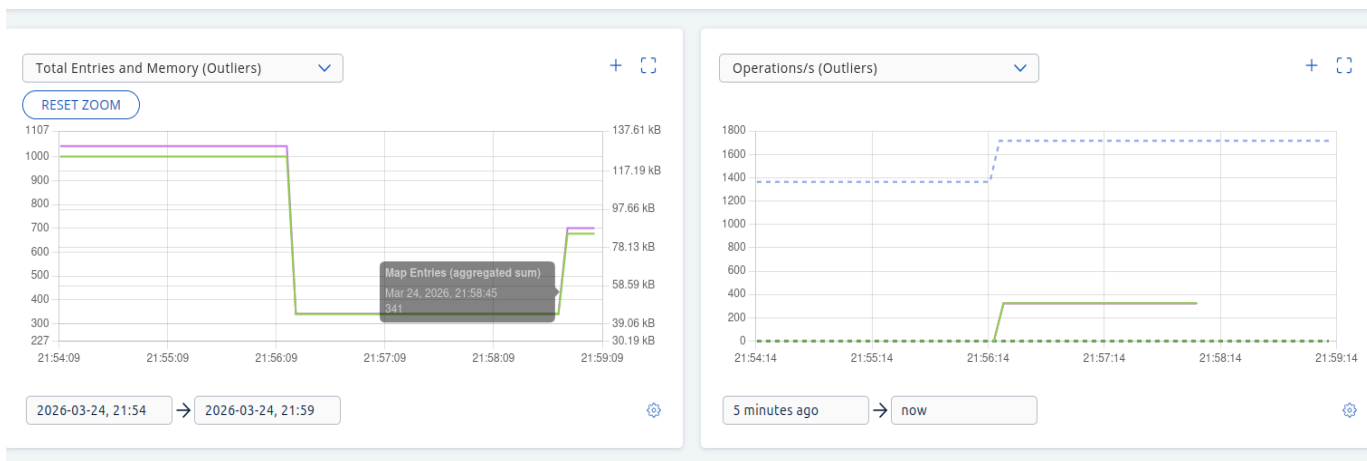


test-map 

CLEAR DATA

MAP BROWSER 

MAP CONFIG 



Map Statistics (In-Memory Format: BINARY)

RESET TIME

1 minute ago

→ now

Default View

Member ^	Entries ^	Gets ^	Puts ^	Removals ^	Sets ^	Entry Memory ^	Events ^	
172.18.0.3:5701	678	0	1,717	0	0	87.99 kB	0	1
TOTAL	678	0	1,717	0	0	87.99 kB	0	1

As you can see we lost around $\frac{1}{3}$ of all data which is a very high loss if you think in the perspective of some product.

To fix this and ensure no data is lost, we must change the Hazelcast configuration (in the `hazelcast.yaml` file or in the code) to set the `backup-count` to 2. This means every piece of data will have two backups, effectively storing a copy on all three nodes in our cluster.

For tasks 4 5 and 6 they are all implemented in one file `task_locks.py`
Here you can see the result of running this file

```
(.venv) bubuntu@bubuntu-computer:~/CLionProjects/Microservices$  
python3 task_locks.py
```

Starting test: No Lock

No Lock finished in 8.98 seconds.

Final value (No Lock): 13046

Starting test: Pessimistic Lock

Pessimistic Lock finished in 82.63 seconds.

Final value (Pessimistic): 30000

Starting test: Optimistic Lock

Optimistic Lock finished in 14.25 seconds.

Final value (Optimistic): 30000

Distributed Map without locks. As seen, the final value is NOT 30000. Because the 3 clients were reading and writing at the exact same time without waiting for each other, they overwrote each other's work and data was lost, basic data race case.

When using Pessimistic and Optimistic locks, the data was perfectly protected. Both tests successfully reached the correct final value of 30,000.

But we can see that Pessimistic lock is 5 times slower than Optimistic. Optimistic locking was significantly faster because it doesn't require sending expensive lock/unlock commands over the network, and it allows all three clients to operate concurrently instead of forcing them to wait in a blocked queue.

Tasks 8

```
(.venv) bubuntu@bubuntu-computer:~/CLionProjects/Microservices$  
python3 task_queue.py
```

Produced: 1

Consumer 1 read: 1

Produced: 2

Consumer 2 read: 2

Produced: 3

Produced: 4

Produced: 5

Produced: 6

Produced: 7
Produced: 8
Produced: 9
Produced: 10
Produced: 11
Produced: 12
Produced: 13
Consumer 1 read: 3
Consumer 2 read: 4
Produced: 14
Produced: 15
Consumer 1 read: 5
Consumer 2 read: 6
Produced: 16
Produced: 17
Consumer 1 read: 7
Consumer 2 read: 8
Produced: 18
Produced: 19
Consumer 1 read: 9
Consumer 2 read: 10
Produced: 20
Consumer 1 read: 11
Produced: 21
Consumer 2 read: 12
Produced: 22
Consumer 1 read: 13
Produced: 23
Consumer 2 read: 14
Produced: 24
Consumer 1 read: 15
Produced: 25
Consumer 2 read: 16
Produced: 26
Consumer 1 read: 17
Produced: 27
Consumer 2 read: 18

It continues to work like that for all 100 numbers

We can clearly see that 1 producer adding numbers, and 2 consumers taking turns reading them immediately. They share the work.

```
(.venv) bubuntu@bubuntu-computer:~/CLionProjects/Microservices$ python3 task_queue.py
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Produced: 6
Produced: 7
Produced: 8
Produced: 9
Produced: 10
```

When the consumers are turned off, the producer stops at 10. Because the bounded queue has a max-size of 10, the producer is blocked and must wait for space to open up.